

Lab Experiments Record

Experiment 1

Question: Create a department page using the following HTML tags. a) Text Formatting tags b) Links c) Images d) Tables e) list f) frames

Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Computer Science Department</title>
</head>
<body>
  <!-- Heading -->
  <h1 align="center">Computer Science Department</h1>
  <!-- Text Formatting -->
  <h2>Text Formatting Tags</h2>
  <b>Bold Text</b><br>
  <i>Italic Text</i><br>
  <u>Underlined Text</u><br>
  <strong>Strong Text</strong><br>
  <em>Emphasized Text</em><br>
  <mark>Highlighted Text</mark><br><br>
  <!-- Link -->
  <h2>Department Website</h2>
  <a href="https://www.google.com" target="_blank">
    Visit Website
  </a>
  <br><br>
  <!-- Image -->
  <h2>Department Image</h2>
  
  <br><br>
  <!-- Table -->
  <h2>Faculty Details</h2>
  <table border="1">
    <tr>
      <th>ID</th>
      <th>Name</th>
      <th>Subject</th>
    </tr>
    <tr>
      <td>101</td>
      <td>Ajay</td>
      <td>Java</td>
    </tr>
    <tr>
      <td>102</td>
      <td>Rahul</td>
      <td>Python</td>
    </tr>
  </table>

```

```

</table>
<br>
<!-- List -->
<h2>Courses Offered</h2>
<ul>
  <li>BCA</li>
  <li>MCA</li>
  <li>B.Tech</li>
</ul>
<ol>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ol>
<!-- Frame -->
<h2>Frame Example</h2>
<iframe
  src="https://www.wikipedia.org"
  width="500"
  height="250">
</iframe>
</body>
</html>

```

Experiment 2

Question: Write JavaScript to validate the following fields of the Registration page. • First Name (Name should contains alphabets and the length should not be less than 6 characters). • Password (Password should not be less than 6 characters length). • E-mail id (should not contain any invalid and must follow the standard pattern name@domain.com) • Mobile Number (Phone number should contain 10 digits only). • Last Name and Address (should not be Empty).

Code:

```

<!DOCTYPE html>
<html>
<head>
<title>Registration Form</title>
<script>
function validate() {
var fname = document.f1.fname.value;
var lname = document.f1.lname.value;
var pass = document.f1.pass.value;
var email = document.f1.email.value;
var phone = document.f1.phone.value;
var addr = document.f1.addr.value;
if(fname.length < 6){
alert("First Name must be 6 characters");
return false;
}
if(lname==""){
alert("Enter Last Name");
return false;
}
if(pass.length < 6){
alert("Password must be 6 characters");
return false;
}

```

```

}
if(email.indexOf("@")==-1 || email.indexOf(".")==-1){
alert("Enter Valid Email");
return false;
}
if(phone.length!=10 || isNaN(phone)){
alert("Enter 10 Digit Mobile Number");
return false;
}
if(addr==""){
alert("Enter Address");
return false;
}
alert("Registration Successful");
return true;
}
</script>
</head>
<body>
<h2>Registration Form</h2>
<form name="f1" onsubmit="return validate()">
First Name:
<input type="text" name="fname"><br><br>
Last Name:
<input type="text" name="lname"><br><br>
Password:
<input type="password" name="pass"><br><br>
Email:
<input type="text" name="email"><br><br>
Mobile Number:
<input type="text" name="phone"><br><br>
Address:
<textarea name="addr"></textarea><br><br>
<input type="submit" value="Register">
</form>
</body>
</html>

```

Experiment 3

Question: a) Develop a ReactJs Application which demonstrate the working of Class component to display Employee details like Sname, Sid, Dept, Age. Apply inline CSS styling for the same b) Develop a program to handle user events c) Develop a React application that includes multiple components. Understand and implement the React component lifecycle methods.

Code:

```

(A)import React, { useState } from "react";
function App() {
  const [name, setName] = useState("");
  const [id, setId] = useState("");
  const [dept, setDept] = useState("");
  const [age, setAge] = useState("");
  const show = () => {
    if (name === "")
      alert("Enter Name");
    else if (id === "")

```

```

        alert("Enter ID");
    else if (dept === "")
        alert("Enter Department");
    else if (age === "")
        alert("Enter Age");
    else
        alert("Employee Details Added");
};
return (
    <div style={{border:"2px solid black", width:"250px", padding:"15px"}}>
        <h2>Employee Details</h2>
        <input
            type="text"
            placeholder="Name"
            onChange={(e) => setName(e.target.value)}
        />
        <br /><br />
        <input
            type="text"
            placeholder="ID"
            onChange={(e) => setId(e.target.value)}
        />
        <br /><br />
        <input
            type="text"
            placeholder="Department"
            onChange={(e) => setDept(e.target.value)}
        />
        <br /><br />
        <input
            type="text"
            placeholder="Age"
            onChange={(e) => setAge(e.target.value)}
        />
        <br /><br />
        <button onClick={show}>Display</button>
        <h3>Name : {name}</h3>
        <h3>ID : {id}</h3>
        <h3>Department : {dept}</h3>
        <h3>Age : {age}</h3>
    </div>
);
}
export default App;
(b)import React from "react";
function App() {
    function message() {
        alert("Button Clicked");
    }
    return (
        <div>
            <h2>User Event Example</h2>
            <button onClick={message}>
                Click Me
            </button>
        </div>
    );
}
}

```

```

export default App;
(c)import React, { Component } from "react";
class LifeCycle extends Component {
  state = { count: 0 };
  componentDidMount() {
    alert("Mounted");
  }
  componentDidUpdate() {
    alert("Updated");
  }
  render() {
    return (
      <div>
        <h2>{this.state.count}</h2>
        <button onClick={() => this.setState({ count: this.state.count + 1 })}>
          Click
        </button>
      </div>
    );
  }
}
export default LifeCycle;

```

Experiment 4

Question: Create and display an order detail list for any 3 items which will include the product name, product description, price and quantity. When an onClick event is triggered on a button the quantity has to increase and the total cost along with Order date and Expected delivery Date, has to be calculated and displayed with the update of items quantity. Order Date: 05/10/2022 (Note - Current date in dd/mm/yyyy) Expected Delivery Date: 08/10/2022 (Note - Current Date + 3 days)

Code:

```

import React, { useState } from "react";

function Orders() {
  const [q1, setQ1] = useState(1);
  const [q2, setQ2] = useState(1);
  const [q3, setQ3] = useState(1);

  const total = q1 * 90000 + q2 * 80000 + q3 * 65000;

  const order = new Date();
  const delivery = new Date();
  delivery.setDate(order.getDate() + 3);

  return (
    <div>
      <h2>Order Details</h2>

      <p>MAC Book - ₹90000</p>
      <p>Qty: {q1}</p>
      <button onClick={() => setQ1(q1 + 1)}>+</button>

      <hr />
    </div>
  );
}

```

```

    <p>Dell Core i7 - ₹80000</p>
    <p>Qty: {q2}</p>
    <button onClick={() => setQ2(q2 + 1)}>+</button>

    <hr />

    <p>HP - ₹65000</p>
    <p>Qty: {q3}</p>
    <button onClick={() => setQ3(q3 + 1)}>+</button>

    <hr />

    <h3>Total: ₹{total}</h3>
    <p>Order Date: {order.toString()}</p>
    <p>Delivery Date: {delivery.toString()}</p>
  </div>
);
}

export default Orders;

```

Experiment 5

Question: Build a basic registration form in React where users can enter first name, last name, and email. Post-registration, an error message is displayed for each field if validation fails else a success message is shown.

Code:

```

import React, { useState } from "react";

function App() {
  const [fname, setFname] = useState("");
  const [lname, setLname] = useState("");
  const [email, setEmail] = useState("");
  const [msg, setMsg] = useState("");

  const register = () => {
    if (fname === "")
      setMsg("First Name is required");
    else if (lname === "")
      setMsg("Last Name is required");
    else if (email === "")
      setMsg("Email is required");
    else
      setMsg("Registration Successful");
  };

  return (
    <div>
      <h2>Registration Form</h2>

      <input
        type="text"

```

```

        placeholder="First Name"
        onChange={(e) => setFname(e.target.value)}
    />
    <br /><br />

    <input
        type="text"
        placeholder="Last Name"
        onChange={(e) => setLname(e.target.value)}
    />
    <br /><br />

    <input
        type="email"
        placeholder="Email"
        onChange={(e) => setEmail(e.target.value)}
    />
    <br /><br />

    <button onClick={register}>Register</button>

    <h3>{msg}</h3>
</div>
);
}

export default App;

```

Experiment 6

Question: Build a REST application for Library Management using NodeJS & Express JS.

Code:

```

npm init -y
npm install express
node server.js

const express = require("express");
const app = express();

app.use(express.json());

let books = [
  { id: 1, title: "Java", author: "James" },
  { id: 2, title: "Python", author: "Guido" }
];

// GET All Books
app.get("/books", (req, res) => {
  res.send(books);
});

// GET Book by ID
app.get("/books/:id", (req, res) => {

```

```

    res.send(books.find(book => book.id == req.params.id));
  });

// ADD Book
app.post("/books", (req, res) => {
  books.push({
    id: books.length + 1,
    title: req.body.title,
    author: req.body.author
  });
  res.send("Book Added");
});

// UPDATE Book
app.put("/books/:id", (req, res) => {
  let book = books.find(book => book.id == req.params.id);

  book.title = req.body.title;
  book.author = req.body.author;

  res.send("Book Updated");
});

// DELETE Book
app.delete("/books/:id", (req, res) => {
  books = books.filter(book => book.id != req.params.id);
  res.send("Book Deleted");
});

app.listen(3000, () => console.log("Server Started"));

```

Experiment 7

Question: Perform CRUD operations with NodeJS with MongoDB.

Code:

```

npm install mongoose
node app.js

const mongoose = require("mongoose");

// Connect MongoDB
mongoose.connect("mongodb://localhost:27017/student",)
// .then(() => console.log("Connected"))
// .catch(err => console.log(err));

// Schema
const StudentSchema = new mongoose.Schema({
  name: String,
  age: Number,
  course: String
});

// Model

```

```

const Student = mongoose.model("Student", StudentSchema);

// CRUD
async function performCRUD() {

  // CREATE
  await Student.insertMany([
    { name: "Rahul", age: 24, course: "MCA" },
    { name: "Anu", age: 22, course: "MBA" },
    { name: "Kiran", age: 23, course: "MSc" }
  ]);
  console.log("Students Added");

  // READ
  const students = await Student.find();
  console.log(students);

  // UPDATE
  await Student.updateMany(
    { name: "Ajay" },
    { $set: { age: 25 } }
  );
  console.log("Student Updated");

  // DELETE
  await Student.deleteOne({ name: "Rahul" });
  console.log("Student Deleted");

  mongoose.connection.close();
}

performCRUD();

```